EMPLOYEE MANAGEMENT AND PAYROLL SYSTEM

A Python Tkinter and MySQL Project Report

Submitted by: _____

Course / Department: _____

Institution: _____

Academic Year: 2026–2027

EMPLOYEE MANAGEMENT AND PAYROLL SYSTEM

Abstract

- The Employee Management and Payroll System is a Python desktop application developed using Tkinter and MySQL. It provides user registration and login, employee-detail entry, salary-payment entry, payment-history viewing, and a payroll summary report.
- The system stores user accounts, employee information, and salary transactions in a MySQL database.
- It calculates net salary using Basic Salary + Allowance − Deduction and displays the total number of employees, payment records, and net salary paid.

Introduction

- Employee information and salary payments are important administrative activities. Manual records can take time to maintain, calculations may be repeated, and retrieving old payment details can be difficult.
- This project provides a graphical desktop application to organize employee and payroll information.
- The supplied program uses Python Tkinter for the interface and MySQL for storage.
- Its modules cover registration, login, dashboard, employee management, salary payment, payment history, and payroll reporting.

Problem Statement

- Maintaining employee details and payroll transactions manually may lead to duplicated work, calculation errors, delayed retrieval of payment history, and poorly organized records.
- A centralized system is needed to store employee information and payment transactions and provide simple screens for common operations.

Objectives

1. Develop an employee management and payroll application using Python Tkinter and MySQL.
2. Provide registration and login screens.
3. Store employee name, phone, email, department, designation, and basic salary.
4. Record salary payments by employee and payment month.
5. Calculate net salary from basic salary, allowance, and deduction.
6. Display payment history in a table.
7. Generate a summary of employees, payments, and total net salary paid.
8. Reduce repetitive manual work and improve organization of payroll data.

Scope of the Project

- **User registration**: create an account with username, password, and password confirmation.
- **Login**: verify entered credentials against the users table.
- **Employee management**: enter and save employee name, phone, email, department, designation, and basic salary.

- **Salary payment**: enter employee ID, payment month, basic salary, allowance, and deduction; validate the employee and calculate net salary.
- **Payment history**: show payment ID, employee ID, month, salary components, net salary, and payment date.
- **Payroll report**: display total employees, total payments, and the sum of net salaries.
- **Database**: use the employee_payment database and its users, employees, and payments tables.

Technologies Used

Technology	Purpose
Python 3	Main programming language.
Tkinter	Creates windows, labels, entry fields, and buttons.
MySQL	Stores user accounts, employee details, and payment records.
mysql.connector	Connects Python to MySQL and runs SQL queries.
ttk.Treeview	Displays payment history in tabular form.
IDLE	Suitable development environments.

System Requirements

A. Hardware:

Component	Minimum Requirement
Processor	Dual-core processor
RAM	4 GB (Recommended)
Hard Disk	200 MB free space
Keyboard	Standard keyboard
Monitor	Standard display

B. Software:

Component	Requirement
Operating System	Windows / Linux
Programming Language	Python 3
Database	MySQL Server
Library	mysql-connector-python
IDE	IDLE
Storage	MySQL Database

Database Design

The database is named employee_payment and contains three tables:

- **users**: id (auto-increment primary key), username (unique and required), and password (required).
    - It supports registration and login.

- **employees**: id (auto-increment primary key), name (required), phone, email, department, designation, and basic_salary.
    - It stores employee information.
- **payments**: id (auto-increment primary key), employee_id (foreign key referencing employees.id), payment_month, basic_salary, allowance, deduction, net_salary, and payment_date.
    - It stores payroll transactions.
- The foreign key links each payment to an employee record. The supplied code uses parameterized SQL statements for its database operations.

Working of the System

The Employee Management and Payroll System works through the following steps:

Step 1: Create the database

The system uses employee_payment database with users, employees, and payments tables.

Step 2: Install and configure

Install Python and mysql-connector-python, then configure database connection in database.py.

Step 3: Start the application

The user runs the main Python file to open the login screen.

Step 4: Register new user

If required, open Register and create an account. The registration form checks required fields and matching passwords.

Step 5: Login verification

Enter the registered credentials. If they match a record, the dashboard opens.

Step 6: Employee Management

Use Employee Management to enter and save employee details.

Step 7: Salary Payment

Use Salary Payment to enter employee ID, month, basic salary, allowance, and deduction. The system checks that the employee exists and calculates net salary.

Step 8: Payment History

Open Payment History to view saved transactions.

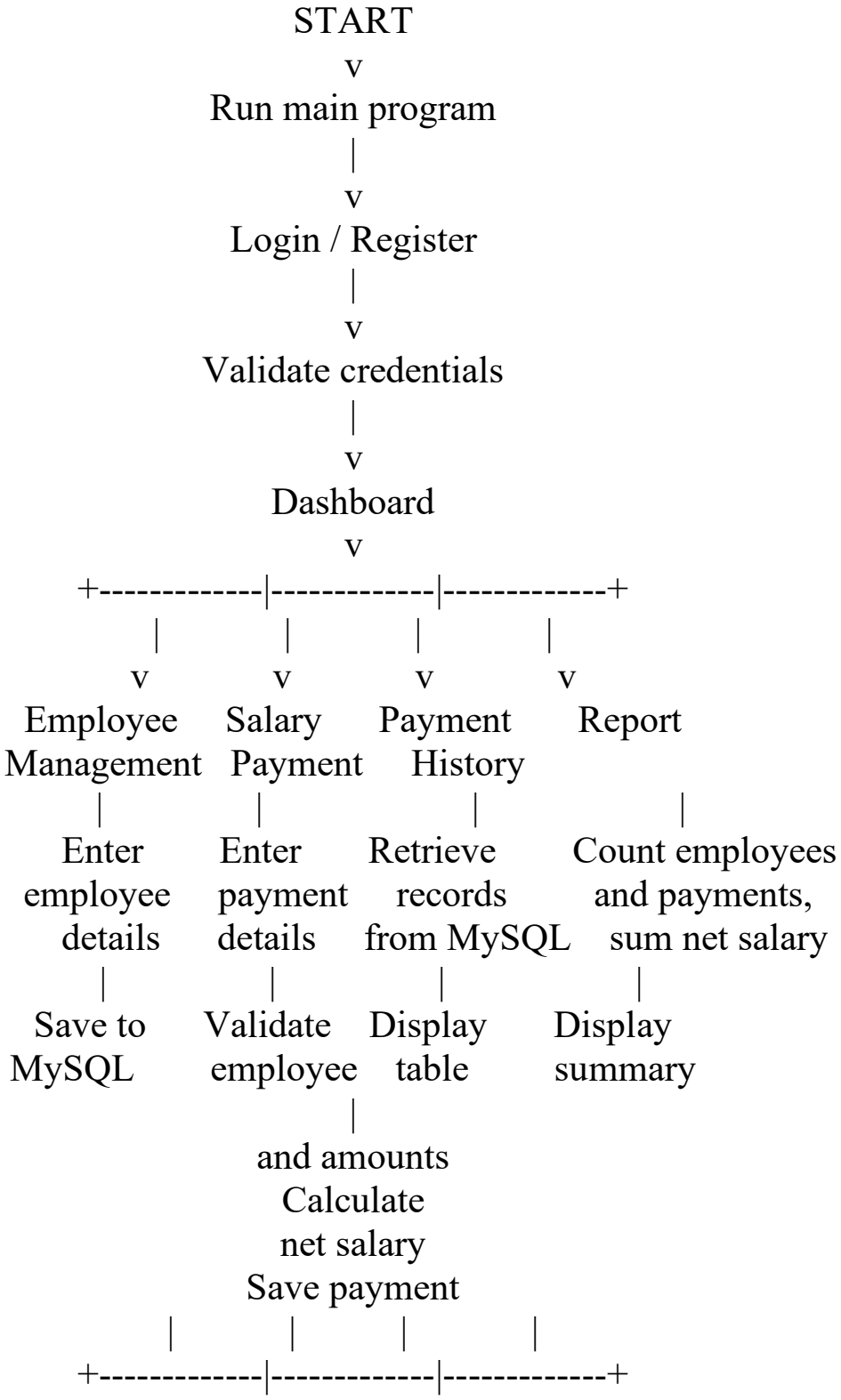
Step 9: View Report

Open Report to view employee count, payment count, and total net salary paid.

Step 10: Logout

Select Logout to close the dashboard and return to the login window.

System Flow


START
Run main program
Login / Register
Validate credentials
Dashboard
Employee Management
Salary Payment
Payment History
Report
Enter employee details
Enter payment details
Retrieve records from MySQL
Count employees and payments, sum net salary
Save to MySQL
Validate employee
Display table
Display summary
and amounts
Calculate net salary
Save payment

|
v
Return to Dashboard
|
v
Logout
|
v
Return to Login / End

Algorithm

1. Start the application and import the required Tkinter and MySQL connector modules.
2. Connect to the employee_payment database.
3. Display the login window and registration option.
4. For registration, validate the username, password, and confirmation, then insert the user record.
5. For login, query the users table with the entered credentials.
6. If login succeeds, open the dashboard; otherwise, show an error message.
7. For employee entry, validate the name and salary, then insert the employee record.
8. For salary payment, validate numeric fields and check that the employee ID exists.
9. Calculate net **salary = basic salary + allowance − deduction**, then save the payment with the current date.
10. For payment history, retrieve payment rows and display them in a Treeview.
11. For the report, count employees, count payments, and sum net_salary.
12. Display database errors through message boxes and close cursors and connections after operations.
13. On logout, close the dashboard and show the login window.

Salary Calculation

Formula:

Net Salary = Basic Salary + Allowance − Deduction.

Example:

Basic Salary = ₹25,000; Allowance = ₹2,000; Deduction = ₹500.

Net Salary = ₹25,000 + ₹2,000 − ₹500 = ₹26,500.

The supplied payment code rejects negative amounts and prevents a deduction from making net salary negative.

Source Code

The following Python program implements the employee management and payroll System. It provides user registration and login, employee-detail entry, salary-payment entry, payment-history viewing, and a payroll summary report.

File name: python database.py

```
import mysql.connector

def get_connection():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="Hemalatha@1407",
```

```
        database="employee_payment"
    )
```

File name: main.py

```
from login import start_login

start_login()
```

File name: register.py

```
from tkinter import *
from tkinter import messagebox
from database import get_connection


def open_register():

    root = Toplevel()
    root.title("Employee Payment System - Register")
    root.geometry("400x350")

    Label(root, text="REGISTER",
          font=("Arial", 20, "bold")).pack(pady=20)

    Label(root, text="Username").pack()
```

```
username = Entry(root, width=30)
username.pack(pady=5)

Label(root, text="Password").pack()
password = Entry(root, width=30, show="*")
password.pack(pady=5)

Label(root, text="Confirm Password").pack()
confirm = Entry(root, width=30, show="*")
confirm.pack(pady=5)

def register():

    user = username.get()
    pwd = password.get()
    cpwd = confirm.get()

    if user == "" or pwd == "":
        messagebox.showerror("Error", "All fields are required")
        return

    if pwd != cpwd:
        messagebox.showerror("Error", "Passwords do not match")
        return

    try:
```

```
        con = get_connection()
        cur = con.cursor()

        cur.execute(
            "INSERT INTO users(username,password) VALUES(%s,%s)",
            (user, pwd)
        )

        con.commit()

        messagebox.showinfo(
            "Success",
            "Registration successful"
        )

        root.destroy()

    except Exception as e:
        messagebox.showerror("Error", str(e))

    finally:
        try:
            cur.close()
            con.close()
        except:
            pass
```

```
    Button(root,
           text="REGISTER",
           width=20,
           command=register).pack(pady=20)
```

File name: login.py

```
from tkinter import *
from tkinter import messagebox
from database import get_connection
from register import open_register
from dashboard import open_dashboard



def start_login():

    root = Tk()
    root.title("Employee Payment System - Login")
    root.geometry("400x400")

    Label(
        root,
        text="EMPLOYEE PAYMENT SYSTEM",
        font=("Arial", 18, "bold")
    ).pack(pady=30)
```

```
Label(
    root,
    text="Username",
    font=("Arial", 12)
).pack()

username = Entry(root, width=30)
username.pack(pady=8)

Label(
    root,
    text="Password",
    font=("Arial", 12)
).pack()

password = Entry(root, width=30, show="*")
password.pack(pady=8)

def login():

    user = username.get()
    pwd = password.get()

    if user == "" or pwd == "":
        messagebox.showerror(
```

```
        "Error",
        "Enter username and password"
    )
    return

try:

    con = get_connection()
    cur = con.cursor()

    cur.execute(
        """
        SELECT * FROM users
        WHERE username=%s AND password=%s
        """,
        (user, pwd)
    )

    result = cur.fetchone()

    cur.close()
    con.close()

    if result:

        messagebox.showinfo(
```

```
                "Login",
                "Login successful"
            )

            # Hide login window
            root.withdraw()

            # Open dashboard
            open_dashboard(root, user)

        else:

            messagebox.showerror(
                "Login Failed",
                "Invalid username or password"
            )

    except Exception as e:

        messagebox.showerror(
            "Database Error",
            str(e)
        )

Button(
    root,
```

```
        text="LOGIN",
        width=20,
        height=2,
        command=login
    ).pack(pady=20)

    Button(
        root,
        text="REGISTER",
        width=20,
        height=2,
        command=open_register
    ).pack()

    root.mainloop()


if __name__ == "__main__":
    start_login()
```

File name: dashboard.py

```
Button(
        frame,
        text="REPORT",
        width=25,
```

```
        height=2,
        command=lambda: open_report(dashboard)
    ).grid(row=1, column=1, padx=20, pady=15)

    # Logout function
    def logout():
        dashboard.destroy()
        login_window.deiconify()

    Button(
        dashboard,
        text="LOGOUT",
        width=20,
        height=2,
        command=logout
    ).pack(pady=30)

    # Handle window close
    dashboard.protocol("WM_DELETE_WINDOW", logout)

    dashboard.deiconify()
    dashboard.lift()
```

File name: employee.py

```
from tkinter import *
```

```
from tkinter import messagebox


def open_employee(parent):
    win = Toplevel(parent)
    win.title("Employee Management")
    win.geometry("600x450")

    Label(
        win,
        text="EMPLOYEE MANAGEMENT",
        font=("Arial", 20, "bold")
    ).pack(pady=20)

    labels = ["Employee Name", "Phone", "Email",
              "Department", "Designation", "Basic Salary"]

    entries = {}

    for item in labels:
        Label(win, text=item).pack()
        entry = Entry(win, width=35)
        entry.pack(pady=3)
        entries[item] = entry

    def save_employee():
```

```
try:
    from database import get_connection

    name = entries["Employee Name"].get().strip()
    if not name:
        messagebox.showerror("Error", "Enter employee name")
        return

    salary = entries["Basic Salary"].get().strip() or "0"
    salary = float(salary)

    con = get_connection()
    cur = con.cursor()

    cur.execute("""
        INSERT INTO employees
        (name, phone, email, department, designation, basic_salary)
        VALUES (%s, %s, %s, %s, %s, %s)
    """, (
        name,
        entries["Phone"].get(),
        entries["Email"].get(),
        entries["Department"].get(),
        entries["Designation"].get(),
        salary
    ))
```

```
            con.commit()
            cur.close()
            con.close()

            messagebox.showinfo("Success", "Employee saved")
            win.destroy()

        except ValueError:
            messagebox.showerror("Error", "Enter a valid salary")
        except Exception as e:
            messagebox.showerror("Database Error", str(e))

    Button(
        win, text="SAVE EMPLOYEE",
        width=20, command=save_employee
    ).pack(pady=15)
```

File name: payment.py

```
from tkinter import *
from tkinter import messagebox
from datetime import date


def open_payment(parent):
```

```
win = Toplevel(parent)
win.title("Salary Payment")
win.geometry("500x500")

Label(
    win, text="SALARY PAYMENT",
    font=("Arial", 20, "bold")
).pack(pady=20)

fields = [
    "Employee ID", "Payment Month",
    "Basic Salary", "Allowance", "Deduction"
]

entries = {}

for field in fields:
    Label(win, text=field).pack()
    entry = Entry(win, width=30)
    entry.pack(pady=4)
    entries[field] = entry

def save_payment():
    con = None
    cur = None
```

```
try:
    employee_id = int(entries["Employee ID"].get())
    month = entries["Payment Month"].get().strip()
    basic = float(entries["Basic Salary"].get())
    allowance = float(entries["Allowance"].get() or 0)
    deduction = float(entries["Deduction"].get() or 0)

    if not month or min(basic, allowance, deduction) < 0:
        messagebox.showerror(
            "Error", "Enter valid payment details"
        )
        return

    net = basic + allowance - deduction

    if net < 0:
        messagebox.showerror(
            "Error", "Deduction cannot exceed gross salary"
        )
        return

    from database import get_connection
    con = get_connection()
    cur = con.cursor()

    cur.execute(
```

```
    "SELECT id FROM employees WHERE id=%s",
    (employee_id,)
)

if cur.fetchone() is None:
    messagebox.showerror("Error", "Employee ID not found")
    return

cur.execute("""
    INSERT INTO payments
    (employee_id, payment_month, basic_salary,
     allowance, deduction, net_salary, payment_date)
    VALUES (%s, %s, %s, %s, %s, %s, %s)
""", (
    employee_id, month, basic,
    allowance, deduction, net, date.today()
))

con.commit()

messagebox.showinfo(
    "Payment Saved",
    f"Net salary: ₹{net:.2f}"
)
win.destroy()
```

```
        except ValueError:
            messagebox.showerror("Error", "Check the numeric values")
        except Exception as e:
            messagebox.showerror("Error", str(e))
        finally:
            if cur:
                cur.close()
            if con:
                con.close()

    Button(
        win, text="SAVE PAYMENT",
        width=20, command=save_payment
    ).pack(pady=20)
```

File name: history.py

```
from tkinter import *
from tkinter import ttk, messagebox


def open_history(parent):
    win = Toplevel(parent)
    win.title("Payment History")
    win.geometry("900x450")
```

```
Label(
    win, text="PAYMENT HISTORY",
    font=("Arial", 20, "bold")
).pack(pady=15)

columns = (
    "ID", "Employee ID", "Month",
    "Basic", "Allowance", "Deduction",
    "Net Salary", "Payment Date"
)

tree = ttk.Treeview(
    win, columns=columns, show="headings"
)

for col in columns:
    tree.heading(col, text=col)
    tree.column(col, width=105)

tree.pack(fill=BOTH, expand=True, padx=10, pady=10)

try:
    from database import get_connection
    con = get_connection()
    cur = con.cursor()
```

```
        cur.execute("""
            SELECT id, employee_id, payment_month,
                   basic_salary, allowance, deduction,
                   net_salary, payment_date
            FROM payments ORDER BY id DESC
        """)

        for row in cur.fetchall():
            tree.insert("", END, values=row)

        cur.close()
        con.close()

    except Exception as e:
        messagebox.showerror("Database Error", str(e))
```

File name: report.py

```
from tkinter import *
from tkinter import messagebox



def open_report(parent):
    win = Toplevel(parent)
    win.title("Payroll Report")
    win.geometry("500x350")
```

```
    Label(
        win, text="PAYROLL REPORT",
        font=("Arial", 20, "bold")
    ).pack(pady=25)

    try:
        from database import get_connection
        con = get_connection()
        cur = con.cursor()

        cur.execute("SELECT COUNT(*) FROM employees")
        employees = cur.fetchone()[0]

        cur.execute("SELECT COUNT(*) FROM payments")
        payments = cur.fetchone()[0]

        cur.execute("SELECT COALESCE(SUM(net_salary), 0) FROM
payments")
        total = cur.fetchone()[0]
        cur.close()
        con.close()
        Label(win, text=f"Total Employees: {employees}",
              font=("Arial", 14)).pack(pady=10)
        Label(win, text=f"Total Payments: {payments}",
              font=("Arial", 14)).pack(pady=10)
        Label(win, text=f"Total Net Salary Paid: ₹{total:.2f}",
```

```
            font=("Arial", 14)).pack(pady=10)
    except Exception as e:
        messagebox.showerror("Database Error", str(e))
```

How to Run the Program

1. Install Python 3 and MySQL Server.
2. Run the SQL statements from the supplied program document to create the employee_payment database and its three tables.
3. Install the connector package: pip install mysql-connector-python
4. Keep the Python modules in the same project folder. The supplied code refers to database.py, login.py, register.py, dashboard.py, employee.py, payment.py, history.py, report.py, and main.py.
5. Set the correct local MySQL username and password in database.py. Do not publish database passwords in reports or public repositories.
6. Start MySQL Server and run main.py.
7. Register a user if necessary, log in, and use the dashboard.

Successful execution depends on the modules being saved correctly, MySQL running, and the connection settings being valid.

Testing

Testing should verify that valid inputs are saved, invalid inputs are rejected, database records can be retrieved, and the report totals are displayed correctly. The following cases are suggested based on the supplied program; run them locally to confirm actual results.

Test Cases

Test case	Input	Expected Result
TC01	Register with valid username and matching passwords:	• success message and user record saved.
TC02	Register with blank required fields:	• validation error displayed.
TC03	Enter mismatched password confirmation:	• password mismatch error displayed.
TC04	Log in with valid credentials:	• dashboard opens.
TC05	Log in with invalid credentials:	• login failed message displayed.
TC06	Save employee with valid details:	• employee record saved.
TC07	Save employee without a name:	• validation error displayed.
TC08	Enter a non-numeric salary:	• valid-salary error displayed.
TC09	Pay a non-existent employee ID:	• employee ID not found message displayed.
TC11	Enter deduction greater than gross salary:	• Enter deduction greater than gross salary:
TC12	Save a valid payment:	• net salary is calculated and saved.

TC13	Open Payment History:	• saved payment rows appear in the table.
TC14	Open Report:	• employee count, payment count, and total net salary appear.
TC15	Select Logout:	• dashboard closes and login window reappears.

Input and Output

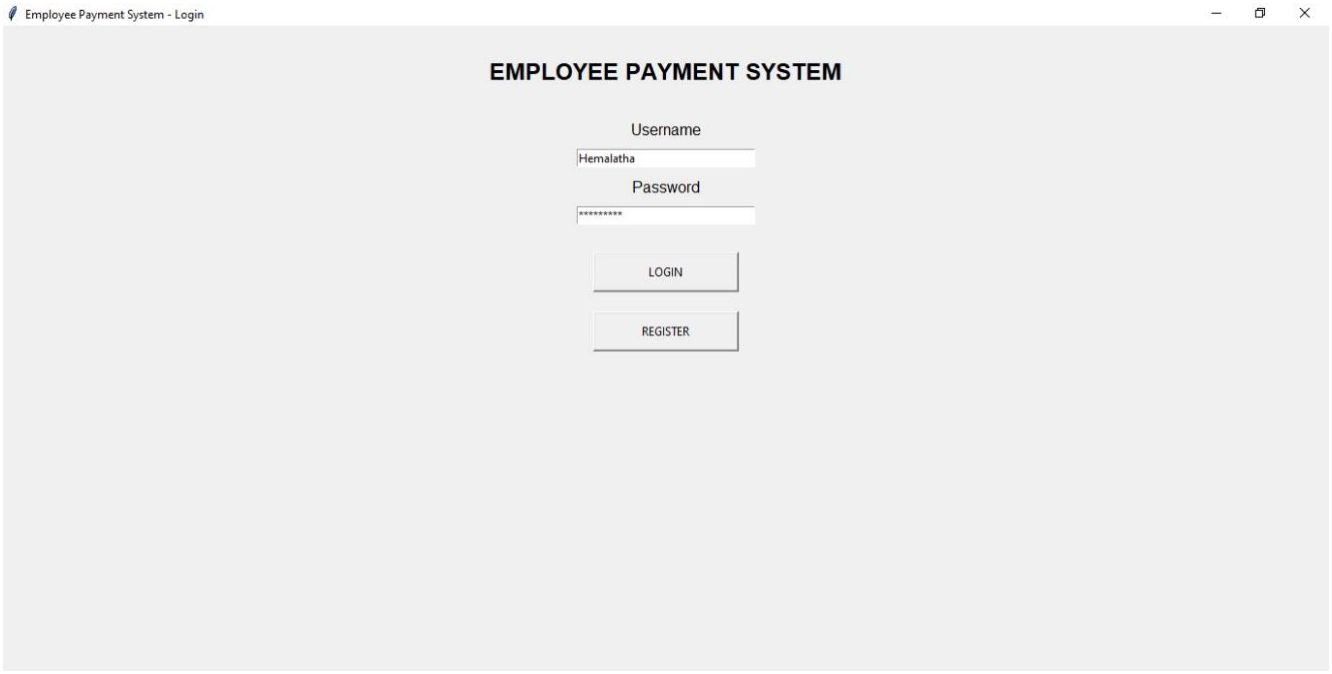
Inputs: username, password, password confirmation, employee name, phone, email, department, designation, basic salary, employee ID, payment month, allowance, and deduction.

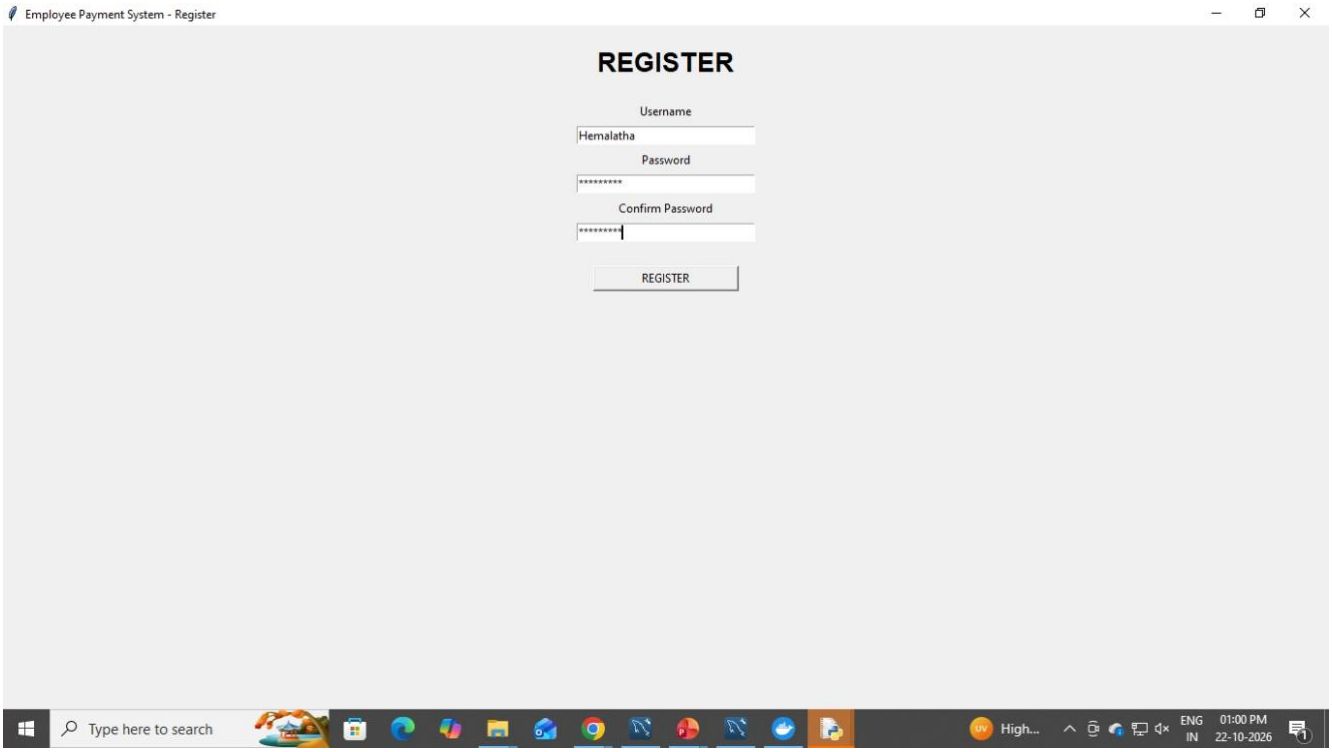
Outputs: registration/login messages, employee-save confirmation, payment confirmation showing net salary, payment-history table, and payroll summary totals.

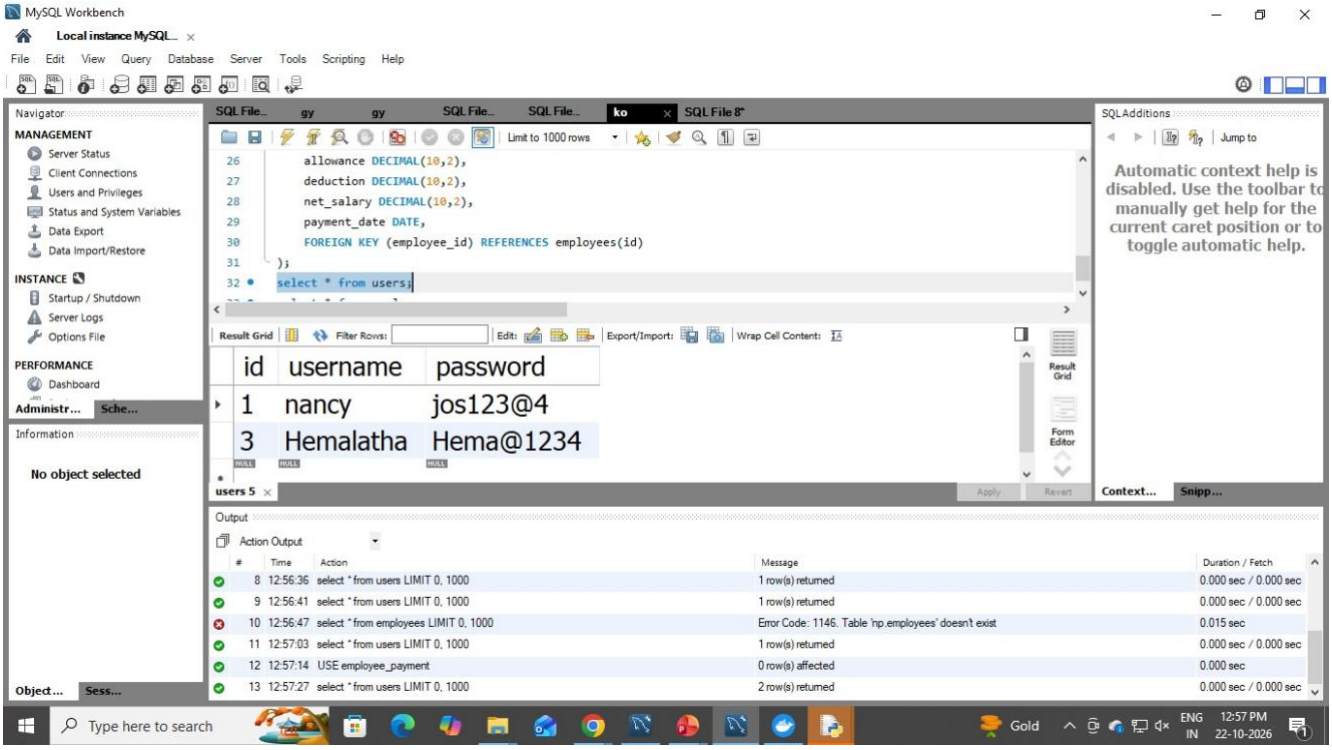
Illustrative payment output:

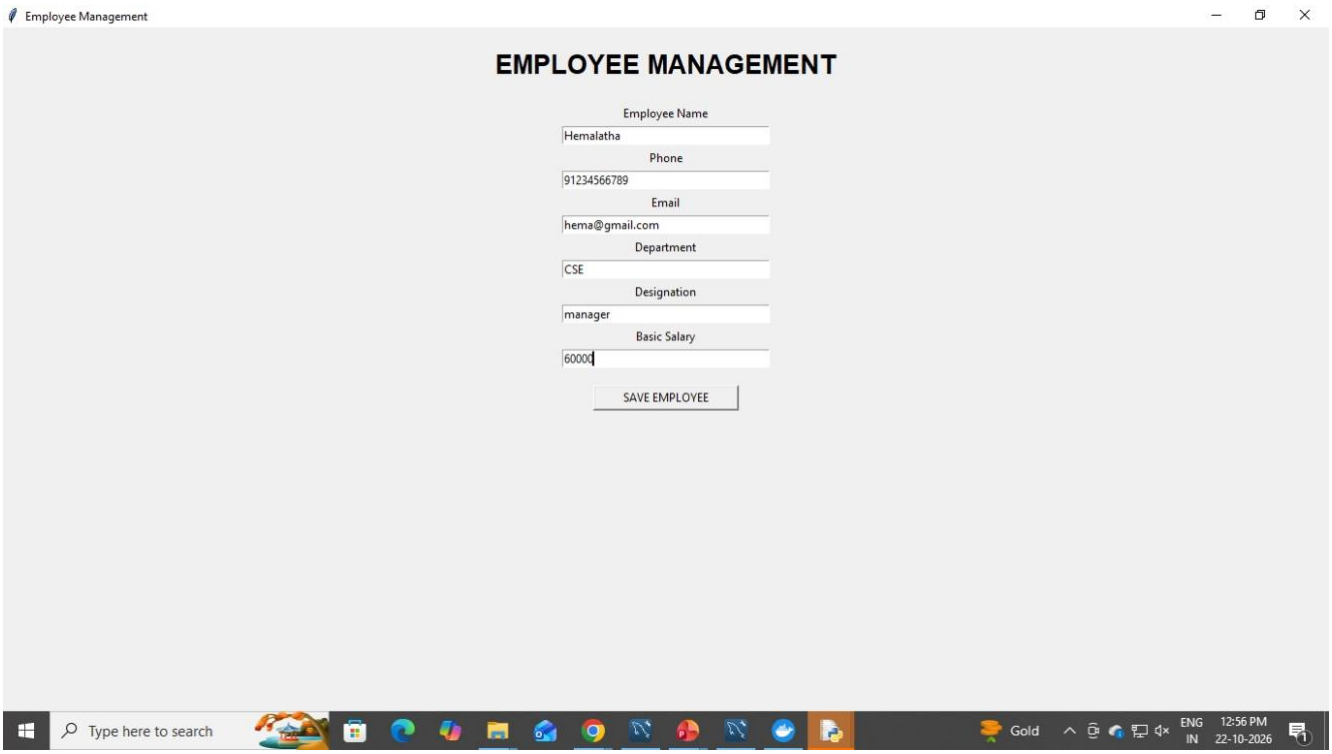
Basic Salary ₹25,000.00 + Allowance ₹2,000.00 - Deduction ₹500.00
= Net Salary ₹26,500.00.

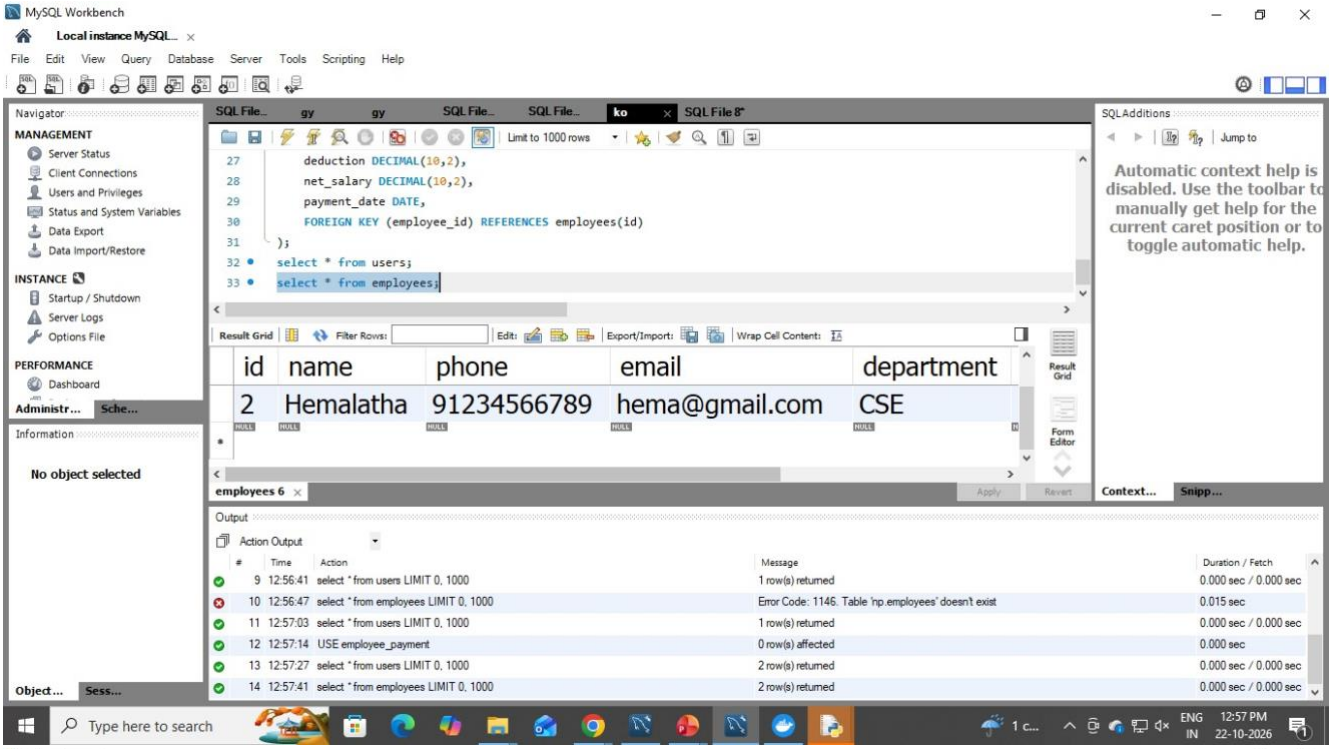
This is an example calculation, not data retrieved from a live database.

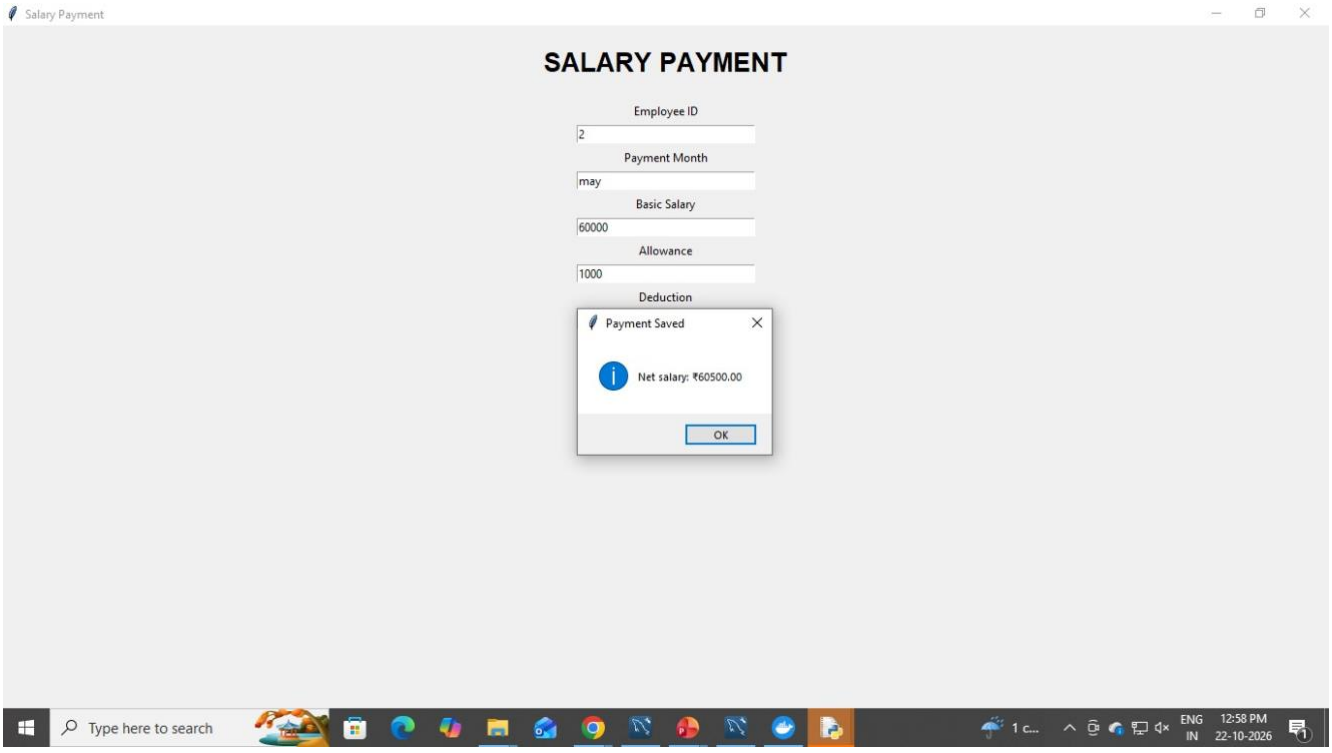

Employee Payment System - Login
EMPLOYEE PAYMENT SYSTEM
Username
Hemalatha
Password
LOGIN
REGISTER


Employee Payment System - Register
REGISTER
Username
Hemalatha
Password
Confirm Password
REGISTER
Type here to search
High...
ENG
IN
01:00 PM
22-10-2026


MySQL Workbench
Local instance MySQL...
File Edit View Query Database Server Tools Scripting Help
Navigator
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variables
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
PERFORMANCE
Dashboard
Information
No object selected
allowance DECIMAL(10,2),
deduction DECIMAL(10,2),
net_salary DECIMAL(10,2),
payment_date DATE,
FOREIGN KEY (employee_id) REFERENCES employees(id)
);
select * from users;
id username password
1 nancy jos123@4
3 Hemalatha Hema@1234
SQL Additions
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.
Output
Action Output
8 12:56:36 select * from users LIMIT 0, 1000 1 row(s) returned 0.000 sec / 0.000 sec
9 12:56:41 select * from users LIMIT 0, 1000 1 row(s) returned 0.000 sec / 0.000 sec
10 12:56:47 select * from employees LIMIT 0, 1000 Error Code: 1146. Table 'hp.employees' doesn't exist 0.015 sec
11 12:57:03 select * from users LIMIT 0, 1000 1 row(s) returned 0.000 sec / 0.000 sec
12 12:57:14 USE employee_payment 0 row(s) affected 0.000 sec
13 12:57:27 select * from users LIMIT 0, 1000 2 row(s) returned 0.000 sec / 0.000 sec
Type here to search
Gold
ENG IN
12:57 PM 22-10-2026


Employee Management
EMPLOYEE MANAGEMENT
Employee Name
Hemalatha
Phone
91234566789
Email
hema@gmail.com
Department
CSE
Designation
manager
Basic Salary
60000
SAVE EMPLOYEE
Type here to search
Gold
ENG IN
12:56 PM 22-10-2026


MySQL Workbench
Local instance MySQL...
File Edit View Query Database Server Tools Scripting Help
Navigator
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variables
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
PERFORMANCE
Dashboard
Information
No object selected
deduction DECIMAL(10,2),
net_salary DECIMAL(10,2),
payment_date DATE,
FOREIGN KEY (employee_id) REFERENCES employees(id)
);
select * from users;
select * from employees;
id name phone email department
2 Hemalatha 91234566789 hema@gmail.com CSE
SQL Additions
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.
Output
Action Output
9 12:56:41 select * from users LIMIT 0, 1000 1 row(s) returned 0.000 sec / 0.000 sec
10 12:56:47 select * from employees LIMIT 0, 1000 Error Code: 1146. Table 'hp.employees' doesn't exist 0.015 sec
11 12:57:03 select * from users LIMIT 0, 1000 1 row(s) returned 0.000 sec / 0.000 sec
12 12:57:14 USE employee_payment 0 row(s) affected 0.000 sec
13 12:57:27 select * from users LIMIT 0, 1000 2 row(s) returned 0.000 sec / 0.000 sec
14 12:57:41 select * from employees LIMIT 0, 1000 2 row(s) returned 0.000 sec / 0.000 sec
Type here to search
ENG IN 12:57 PM 22-10-2026


Salary Payment
SALARY PAYMENT
Employee ID
2
Payment Month
may
Basic Salary
60000
Allowance
1000
Deduction
Payment Saved
Net salary: ₹60500.00
OK
Type here to search
ENG IN 12:58 PM 22-10-2026

Payment History

PAYMENT HISTORY

ID	Employee ID	Month	Basic	Allowance	Deduction	Net Salary	Payment Date
2	2	may	60000.00	1000.00	500.00	60500.00	2026-10-22
1	1	october	30000.00	10000.00	0.00	40000.00	2026-10-21



Payroll Report

PAYROLL REPORT

Total Employees: 2

Total Payments: 2

Total Net Salary Paid: ₹100500.00



Advantages

1. Centralized storage of employee and payroll information in MySQL.
2. A graphical interface with Tkinter windows and buttons.
3. Faster retrieval of previously saved payment details.
4. Automatic net salary calculation.
5. A summary report for employee count, payment count and total net salary paid.
6. Input checks for required values, numeric amounts, employee existence, and invalid salary combinations.
7. A modular structure that separates different application functions.

Limitations and Future Scope

- **Limitations**: The supplied code appears to store and compare passwords as plain text; production software should use salted password hashes. The code shown does not include role-based access,employee editing/deletion, advanced search, payslip PDF generation, or report export. The report is an overall summary rather than a month-wise or department-wise breakdown. Salary details are entered on the payment form rather than automatically loaded from the employee's saved basic salary.
- **Future improvements**: Add secure password hashing and permissions; employee search, update, and delete; load salary from employee records; month-wise and department-wise reports; PDF/Excel export; payslip generation; backups; and audit logs.

Conclusion

- The Employee Management and Payroll System is built using Python Tkinter and MySQL. It allows users to register, login, save employee details, record salary payments, view payment history and payroll summary.This project demonstrates GUI programming, validation, SQL queries and database connectivity. It can be extended with payslip generation and report export in future.